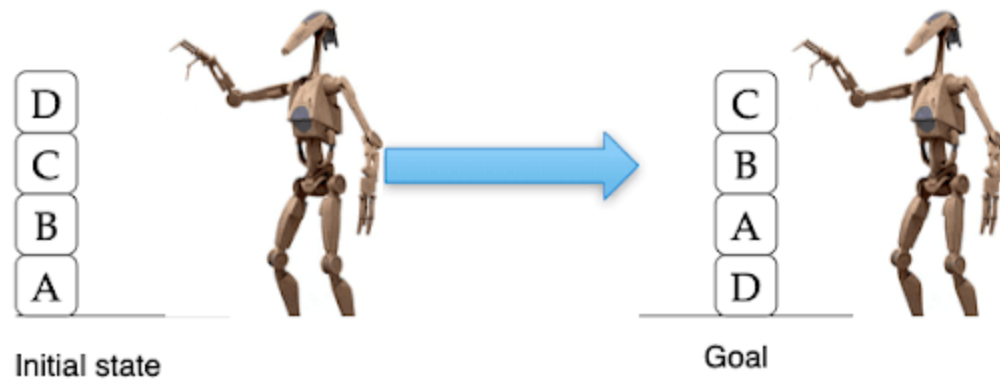


Práctica 1.2 PDDL



David Medina Ruíz

Curso de especialización de Inteligencia Artificial y Big Data

Curso 2025 - 2026

Introducción

En esta práctica debemos programar y encontrar las mejores soluciones para un robot camarero que debe recorrer las distintas habitaciones del restaurante.

El objetivo de esta práctica es que el robot sea capaz de moverse desde una ubicación inicial (el recibidor, por ejemplo) hasta una ubicación objetivo (el aseo, por ejemplo), navegando de forma autónoma a través de las distintas habitaciones del restaurante.

Para resolver el problema, se emplea PDDL (Planning Domain Definition Language).

Ficheros

El proyecto está compuesto por dos ficheros escritos en PDDL, cada uno con una finalidad distinta en la definición de la planificación.

Fichero dominio.pddl

En este fichero se define la estructura y las reglas que debe seguir el robot. Consta de varios componentes:

Declaración del dominio

```
1 (define (domain robot-camarero)
2   (:requirements :typing)
```

Define el nombre del dominio “robot-camarero” y especifica que utilizaremos tipado (:typing) para clasificar objetos.

Tipado de objetos

```
(:types habitacion robot)
```

Se definen dos clases de objetos: habitación, que representa cualquier estancia del restaurante, y robot, que representa al agente autónomo que se desplaza por el entorno.

Predicados

```
(:predicates
  (en ?r - robot ?h - habitacion)
  (conectadas ?h1 - habitacion ?h2 - habitacion)
)
```

Los predicados definen las relaciones entre los objetos:

- "en ?r - robot ?h - habitacion": indica que un robot se encuentra en una habitación.
- "conectadas ?h1 - habitacion ?h2 - habitacion": indica que existen conexiones entre las habitaciones, es decir, una puerta.

Acciones

```
(:action mover
  :parameters (?r - robot ?desde - habitacion ?hacia - habitacion)
  :precondition (and
    (en ?r ?desde)
    (conectadas ?desde ?hacia)
  )
  :effect (and
    (not (en ?r ?desde))
    (en ?r ?hacia)
  )
)
```

Aquí definimos las acciones que el robot puede realizar, en este caso simplemente puede realizar la acción "mover" para desplazarse por las habitaciones:

- Los parámetros son el robot, la habitación de origen y la habitación destino.
- Las precondiciones, o acciones que deben cumplirse para ejecutar la acción, en las que el robot debe estar actualmente en la habitación origen y que exista una conexión entre el origen y destino.

-
- Los efectos, o cómo cambia el mundo tras ejecutar la acción, en la que el robot deja de estar en la habitación de origen y pasa a estar en la habitación destino.

Fichero problema.pddl

En este fichero definimos una configuración concreta del restaurante y una tarea particular que el robot debe resolver.

Declaración del problema

```
(define (problem robot-camarero-habitacion)
  (:domain robot-camarero))
```

Definimos el nombre del problema "robot-camarero-habitacion" y lo vinculamos con el dominio que hemos definido "robot-camarero".

Objetos concretos

```
:objects
  r - robot
  hab1 hab2 aseo pasillo recibidor salon cocina - habitacion
```

Declaramos cada estancia de las distintas clases que vamos a usar, es decir, un robot y siete habitaciones.

Estado inicial

```
(:init
  ;; situamos al robot en una habitación, por ejemplo en el recibidor
  (en r recibidor)

  ;; conexiones con el pasillo
  (conectadas pasillo hab1)
  (conectadas hab1 pasillo)
  (conectadas pasillo hab2)
  (conectadas hab2 pasillo)
  (conectadas pasillo aseo)
  (conectadas aseo pasillo)
  (conectadas pasillo recibidor)
  (conectadas recibidor pasillo)
  (conectadas pasillo salon)
  (conectadas salon pasillo)
  (conectadas pasillo cocina)
  (conectadas cocina pasillo)

  ;; conexiones entre habitaciones
  (conectadas recibidor salon)
  (conectadas salon recibidor)
  (conectadas salon cocina)
  (conectadas cocina salon)
)

(:goal
  (en r aseo)
)
)
```

Con el método (:init) definimos el estado del mundo inicial al comienzo:

- Situamos al robot en una habitación, en este caso, en el recibidor.
- Definimos las conexiones entre las habitaciones. Lo hacemos de forma bidireccional para que el robot pueda circular en ambos sentidos.

Objetivo

```
(:goal
  (en r aseó)
)
```

Con el método (:goal) definimos el objetivo al que el planificador debe llegar. Hemos definido que el robot debe terminar en el aseó

Ejemplo de ejecución

Junto con el [solver.py](#) proporcionado por el profesor, ejecutamos el siguiente comando:

```
C:\Users\David\Desktop\IA y BigData\Modelos de IA\Practica1,2PDDL>python solver.py dominio.pddl problema.pddl plan.txt
```

Obtendremos un fichero txt llamado plan, en el que obtendremos el resultado generado por el planificador.

```
plan.txt
1  (mover r recibidor pasillo)
2  (mover r pasillo aseó)
```

Como podemos observar, en dos movimientos ha cumplido el objetivo.

Vamos a realizar cambios para que esta vez se desplace desde el recibidor hasta la cocina, por ejemplo.

<pre>(:goal (en r cocina))</pre>	<pre>plan.txt 1 (mover r recibidor pasillo) 2 (mover r pasillo cocina)</pre>
-------------------------------------	--

Ha vuelto a encontrar el camino en dos movimientos.

Ahora vamos a cambiar la habitación de origen, en vez del recibidor pondremos la habitación 1 hasta la cocina

```
(:init
;; situamos al robot en
(en r hab1)
```

```
plan.txt
1  (mover r hab1 pasillo)
2  (mover r pasillo cocina)
```

Vuelve a completar el objetivo en 2 pasos.